

Implementation Plan & Development Phases

Product: ExamGuard — Smart Exam Allocation & Identity Verification System **Document Version:** 2.0 (Solo, AI-Assisted Build) **Date:** August 14, 2026

1. Delivery Approach

This plan assumes a **solo, non-technical founder building ExamGuard entirely with AI coding assistance** — no in-house engineering team, no dedicated designer, and no ops staff. The approach is deliberately different from a conventional engineering-team plan: it favors **managed, low-maintenance services over self-hosted infrastructure, small AI-directed build sessions over sprints, and your own manual testing over a QA team**. Your primary job throughout is not writing code — it is directing the AI clearly, verifying what it builds actually works, and making product decisions. The AI (e.g., Claude Code) does the implementation; you do the specifying, checking, and deciding.

The five phases from the original plan are kept, but each is re-scoped for one person working with AI assistance rather than a multi-person team, and the "team" section is replaced with a toolkit of services that remove the need for dedicated backend/DevOps engineers.

2. What Changes When AI Builds It (Read This First)

- **You will not write code line-by-line — you will describe outcomes.** Every build session starts with a clear, specific instruction ("Add a CSV upload screen that validates roll numbers and shows errors inline") rather than a technical spec. The more concrete and example-driven your instructions, the better the output.
- **You must still test every feature yourself before moving on.** AI-generated code can look correct and still have bugs (a wrong seat gets allotted twice, a barcode doesn't expire). Section 6 gives you a non-technical way to verify each phase.
- **Prefer managed services over things you'd have to operate yourself.** Running your own PostgreSQL server, Redis instance, and Node.js backend (as in the original TRD) requires ongoing server administration you're not set up to do solo. This plan substitutes managed, all-in-one platforms that handle hosting, backups, and scaling for you, while keeping the same data model from the Backend Schema document.
- **Keep everything in one place you control.** Use a single GitHub (or similar) repository from day one, even though you're not writing the commits yourself — most AI coding tools can commit for you. This gives you a history to roll back to if a change breaks something.
- **Budget for a small amount of paid help at specific checkpoints,** even though the build itself is solo — see Section 7. A few hours of a freelance developer or security reviewer at the right moment is far cheaper than a data leak of student photos or a seat-collision bug on exam day.

3. Recommended Simplified Stack (for a Solo AI-Assisted Build)

The original TRD's stack (self-hosted NestJS + PostgreSQL + Redis + Kubernetes) assumes a team that can operate infrastructure. For a solo build, substitute managed equivalents that keep the same relational data model and core logic, but remove server administration entirely:

Layer	Original (Team) Stack	Recommended Solo Stack	Why
Backend + Database	Custom NestJS API + self-hosted PostgreSQL	Supabase (managed PostgreSQL + auto-generated APIs + built-in Auth + Storage)	No server to patch or scale yourself; the Backend Schema's tables map directly into Supabase; Row-Level Security replaces hand-written access-control code
Realtime / Live Dashboard	Custom WebSocket layer + Redis pub/sub	Supabase Realtime (built on Postgres replication)	Live dashboard updates with no separate Redis service to run
Scheduled Reveal Job	Custom Redis-backed job queue	Supabase Edge Functions on a cron schedule (or a simple scheduled function on Vercel)	A managed scheduler triggers the T-30 unlock without you operating a queue
Student & Admin Web Apps	Next.js, self-hosted	Next.js on Vercel	One-command deploy, automatic HTTPS, scales without configuration
Invigilator Mobile App	Expo / React Native	Expo / React Native, unchanged — distributed via EAS Build and Expo Go for testing	Already the right choice for a solo, no-app-store-review build/test loop
Photo Storage	S3-compatible object storage	Supabase Storage	Same bucket-and-signed-URL model, one less account to manage
Offline Cache (Invigilator App)	expo-sqlite	expo-sqlite, unchanged	No change needed — this already fits a solo build
Auth	Custom JWT issuance	Supabase Auth	Login, password reset, and session/token handling come built-in instead of hand-rolled

This substitution changes *how* the system is hosted and operated, not *what* it does — every feature, table, and flow in the PRD, App Flow, and Backend Schema documents stays the same. Treat those documents as the source of truth for behavior; treat this stack table as the source of truth for what you'll actually tell the AI to build with.

4. Phase Breakdown (Solo, AI-Directed)

Each phase below is scoped to what one person, working part-time with an AI coding assistant, can realistically direct, test, and sign off on — not what a team could build in the same calendar time. Expect each phase to take longer in elapsed time than the original team-based plan, but require far less of *your* active effort per feature, since the AI is doing the implementation.

Phase 0 — Foundation & Setup (Weeks 1-2)

- Create accounts: GitHub, Supabase, Vercel, Expo/EAS.
- Have the AI scaffold the project: a Next.js app (admin console + student portal) and an Expo app (invigilator scanner), both connected to a new Supabase project.
- Have the AI create the database tables from the Backend Schema document directly in Supabase (students, halls, exams, student_exam_mappings, verification_logs, invigilators, admins).
- Set up basic Supabase Auth for two user types: admin/invigilator accounts and student accounts.
- **Your task:** confirm you can log into an empty admin console and an empty student portal, and that the tables exist in the Supabase dashboard with the right columns.
- **Exit criteria:** you can see all core tables in Supabase, and both web apps load a blank authenticated screen.

Phase 1 — Admin Core & Data Foundation (Weeks 3-7)

- Roster bulk upload (CSV/Excel) with validation and a downloadable error report for bad rows.
- Hall management (create/edit halls with capacity).
- Exam creation (course, date, start/end time).
- Student-exam-hall-seat mapping, both bulk auto-allocation and manual override, using the database's seat-uniqueness constraint so double-booking a seat is impossible even if the AI's application logic has a bug.
- Barcode token generation on mapping creation.
- **Your task:** upload a real (or realistic test) roster, create a hall and an exam, map 10-20 students, and manually confirm in the Supabase table view that no two students share a seat.
- **Exit criteria:** you can go from a raw spreadsheet to a fully seat-mapped exam with generated tokens, entirely through the admin UI, with no manual database editing required.

Phase 2 — Student Portal & Reveal Engine (Weeks 6-10, overlapping Phase 1)

- Student login and timetable view.
- Dynamic, auto-refreshing barcode pass screen.
- Scheduled reveal job (T-30 unlock) and the locked/unlocked hall-seat UI.
- **Your task:** create a test exam scheduled a few minutes in the future, log in as a test student, and watch the hall/seat panel actually

- unlock on its own at the configured time without refreshing the page.
- **Exit criteria:** the locked-to-revealed transition works correctly at least three times in a row with different reveal thresholds (e.g., 2 minutes, 30 minutes) before you trust it for a real exam.

Phase 3 — Invigilator Scanner App (Weeks 9-14, overlapping Phase 2)

- Expo app shell, login, and hall/exam selection.
- Camera-based barcode scanning.
- Verification result screen (green/red/amber states) and manual roll-number search.
- Offline caching of the hall roster and photos, plus a local scan queue that syncs when connectivity returns.
- **Your task:** this is the highest-risk phase to test alone — put your phone in airplane mode, scan 10-15 test students, turn airplane mode off, and confirm every scan appears correctly in Supabase with no duplicates or missing records. Repeat at least twice.
- **Exit criteria:** a full offline-then-sync cycle completes with zero data loss and zero duplicate verification records, verified by manual inspection of the `verification_logs` table.

Phase 4 — Live Dashboard & Hardening (Weeks 13-17, overlapping Phase 3)

- Live attendance dashboard (verified/pending/absent/wrong-hall counts per hall).
- Audit log with filtering and CSV export.
- A security pass focused on the highest-impact risks for a solo, non-security-expert builder: confirm hall/seat data is genuinely withheld from API responses before reveal (not just hidden in the UI), confirm student photo URLs aren't publicly guessable, and confirm a used barcode is rejected on a second scan.
- **Your task:** this security pass is the one point in the project where paid outside help is strongly recommended even for a solo build — see Section 7. At minimum, manually attempt each of the three checks above yourself.
- **Exit criteria:** the three security checks above all pass, and the dashboard updates within a few seconds of a test scan.

Phase 5 — Pilot, Real-World Test & Launch (Weeks 17-20)

- Run one full dry-run exam with a small real or simulated group (colleagues, friends, or a small willing class) in an actual room, using a real phone as the scanner.
- Fix anything that breaks during the dry run before trusting the system with a real exam cycle.
- Prepare a one-page printed instruction sheet for invigilators and a short note for students, since there is no dedicated training team to walk people through it live.
- Keep the manual roll-number search and a paper backup list on hand for the first real exam, in case anything unexpected happens.
- **Exit criteria:** the dry run completes with no lost or duplicated verification records, and you personally feel confident enough to run it for a real exam without a developer standing by.

5. Milestone Timeline

Phase	Weeks (elapsed)	Key Deliverable
Phase 0 — Foundation & Setup	1-2	Accounts, scaffolded apps, database tables live
Phase 1 — Admin Core & Data Foundation	3-7	Roster upload, mapping engine, tokens
Phase 2 — Student Portal & Reveal Engine	6-10	Timetable, dynamic pass, T-30 reveal
Phase 3 — Invigilator Scanner App	9-14	Offline scanning, sync, verification UI
Phase 4 — Dashboard & Hardening	13-17	Live dashboard, security checks
Phase 5 — Pilot & Launch	17-20	Dry run, fixes, first real exam cycle

Total estimated elapsed timeline: **~20 weeks** working solo, part-time, with AI assistance — roughly 4-5 weeks longer than the original team-based estimate, which is expected: the calendar time trades off against not needing to hire, manage, or pay a team. If you can dedicate more hours per week, or if AI build sessions go smoothly with few corrections needed, this can compress; budget extra time in Phase 3 and Phase 4 specifically, since offline-sync correctness and security are the two areas where "looks right" and "is right" differ most, and you are the only one checking.

6. How to Test Each Phase Without Writing Code

You don't need to read code to verify it works — you need repeatable manual checks:

- **Use the Supabase table editor as your source of truth.** After any action in the app (uploading a roster, mapping a seat, scanning a barcode), open the relevant table in Supabase and confirm the row looks right. This catches most bugs before they reach a real exam.
- **Create a small, disposable test dataset** (5-10 fake students, one hall, one exam scheduled a few minutes out) and re-run it after every meaningful change, rather than only testing once at the end of a phase.
- **Write down a simple checklist per phase** (e.g., for Phase 1: "upload roster → see 10 students → map to Hall A → no seat appears twice → each mapping has a token") and re-run that exact checklist after each AI build session, not just at the end.
- **Test the unhappy paths on purpose:** upload a roster with a duplicate roll number, scan the same barcode twice, disconnect the phone mid-scan. These are the scenarios most likely to be wrong in AI-generated code and least likely to be caught by accident.
- **Ask the AI to explain what it changed, in plain language, after each session,** and read that summary before testing — it tells

you exactly what to check.

7. When to Bring In Paid Human Help

A fully solo, AI-built product is realistic for ExamGuard, but a few moments are worth budgeting for a few hours (or a single paid consultation) with someone who has professional experience, rather than relying on AI and self-testing alone:

- **Before Phase 5 (pilot):** a short paid security review (even 2–4 hours from a freelance developer) focused specifically on the three checks in Phase 4 — hall/seat data withholding, photo URL privacy, and barcode single-use enforcement — since these protect real students' personal data and exam integrity.
- **If you plan to scale beyond a single pilot institution:** a brief architecture review before Phase 5, to confirm the Supabase-based stack in Section 3 will hold up at your expected student volume.
- **Legal/compliance review** of how student photos and personal data are stored and retained, appropriate to your institution's data protection obligations — this is a legal question, not a coding one, and outside what an AI assistant or this plan can advise on.

Everything else in this plan — the actual building, testing, and iterating — is designed to be done by you, directing the AI, without needing to hire a team.

8. Risk-Managed Rollout

- Launch with a single small pilot (one class or department) before trusting the system with a full exam cycle, since you are your own QA team and smaller scale means smaller consequences if something is missed.
- Keep the manual roll-number search and a printed paper backup list available for at least the first two or three real exam cycles.
- Be present (or reachable) in person during the first few real exam windows, since there is no on-call engineering team — you are the hypercare coverage.
- Don't add new features between the pilot and the first real launch; use that window only to fix what the pilot revealed.

9. Post-Launch Roadmap (Beyond v1)

- Face-match/liveness-assisted identity verification to further reduce impersonation risk.
- Multi-institution support if you decide to offer ExamGuard to more than one institution.
- Two-way roster sync with an existing student information system, if one exists at your institution.
- A standalone (non-Expo-Go) app build via EAS if you outgrow Expo Go for distribution.
- Simple analytics (no-show rates, hall utilization trends) once you have a few exam cycles of real data.